

Introduction to Software Engineering

Project Proposal

The student team is required to complete the **Project Proposal** documentation for the assigned course project, following the attached template.



Software Engineering Department
Faculty of Information and Technology
University of Science

Table of Contents

Objectives	1
1 Member Contribution Assessment	2
2 Preliminary Problem Statement	3
3 Proposed Solution	4
4 Development Plan	5
5 Human Resources & Costing Plan	6
6 Tools setup	7

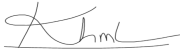
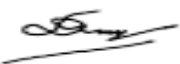
Project Proposal

Objectives

This document focus on the following topics:

- ✓ Completing the Project Proposal document with the following sections:
 - Preliminary Problem Statement
 - Proposed Solution
 - Development Plan
 - Human Resources & Costing Plan
- ✓ Understanding the Project Proposal document.

1 Member Contribution Assessment

ID	Name	Contribution (%)	Signature
23127004	Lê Nhật Khôi	100%	
23127158	Nguyễn Thanh Gia Bảo	100%	
2312765	Nguyễn Hải Đăng	100%	
23127181	Nguyễn Trần Quốc Duy	100%	
23127438	Đặng Trường Nguyên	100%	

2

Preliminary Problem Statement

In recent years, the demand for pet ownership has grown rapidly as pets have become an essential part of modern human life. Pets not only provide companionship and emotional support but also contribute to improving mental health, reducing stress, and enhancing quality of life. Many people now consider pets as members of their families, investing time, care, and financial resources to ensure their well-being. The growing social trend of pet keeping has led to an increasing demand for various animal species, such as dogs, cats, birds, rabbits, hamsters, and ornamental fish, as well as for related products like food, accessories, and healthcare services.

Along with this surge in demand, the pet trading market has expanded significantly both online and offline. However, the current processes of buying and selling pets still face multiple challenges. Information about available pets is often scattered across social media platforms, unverified websites, or small local pet stores, making it difficult for buyers to compare prices, verify authenticity, or assess the health and origin of pets. Additionally, many transactions lack transparency and security, leading to issues such as misinformation, poor after-sale support, and even fraudulent activity. Furthermore, the absence of an integrated online platform causes inconvenience in communication between buyers and sellers, as well as inefficiencies in managing pet listings and user data.

To address these issues, our development team proposes **5Pets**, a web-based application designed to create a safe, convenient, and reliable environment for buying, selling, and managing pet-related activities. The system will consist of two main subsystems:

- Customer Subsystem (Client Side): The user-facing web interface that allows customers to browse, search, and purchase pets or related products.
- Admin Dashboard (Admin Side): The management portal, also known as the Admin Dashboard, is exclusively used by the website owner or system administrator to manage listings, users, and categories within the platform.

When a customer accesses the **5Pets** website, the system will automatically display the main homepage containing categories such as pets, accessories, food, and care services. Users can view detailed information about each pet, including its name, species, breed, gender, age,

health status, vaccination record, and price. Each pet post will include a description, photos, and seller contact details.

Users can register for an account as a buyer, sellers can register for an account as an admin

- A buyer can search for pets by species, price range, or location, and add desired pets to a shopping cart or favorites list.
- A seller can create new listings, upload pet photos, information about availability or pricing, and delete products.

All data related to pets, users, and transactions will be stored in a centralized database managed by the web server. The **Admin Dashboard** provides essential CRUD (Create, Read, Update, Delete) functionalities, allowing administrators to manage core system data efficiently. Through this dashboard, the admin, who is also the host, can create, modify, or remove records related to pets, users, product categories, and orders. This subsystem ensures that data remains organized, up-to-date, and consistent across the platform. Future enhancements may include advanced features such as seller verification, review moderation, and sales reporting. On the other hand, the admin can also modify the logo, information about the shop (phone, address, ...) and the favicon of the website through this Admin Dashboard.

The operating environment of the **5Pets** system will be based on modern web technologies. The application will run on web browsers supporting HTML5, such as Google Chrome, Microsoft Edge, or Mozilla Firefox. The **frontend** will be developed using HTML5, CSS3, and JavaScript (with React frameworks and TailwindCSS Style frameworks) to ensure responsiveness and user interactivity. The **backend** will be implemented using Python (with Flask framework), hosted on the free platform Render/Vercel server. All persistent data will be stored in MongoDB.

The system must comply with standard documentation and security practices, including HTTPS encryption, input validation, and secure password hashing. It will also adhere to **IEEE documentation standards** and be developed following the **Waterfall model**, with clearly defined phases for requirements, design, implementation, testing, and deployment.

In conclusion, the **5Pets** aims to create a centralized and secure platform for buying and selling pets and pet-related products. By providing an intuitive user interface, transparent listing verification, and robust management tools, the system will not only enhance the convenience of online pet trading but also promote responsible pet ownership and community engagement. This preliminary problem statement serves as the foundation for further analysis and detailed specification in subsequent development phases.

3

Proposed Solution

3.1 Software

3.1.1. Features

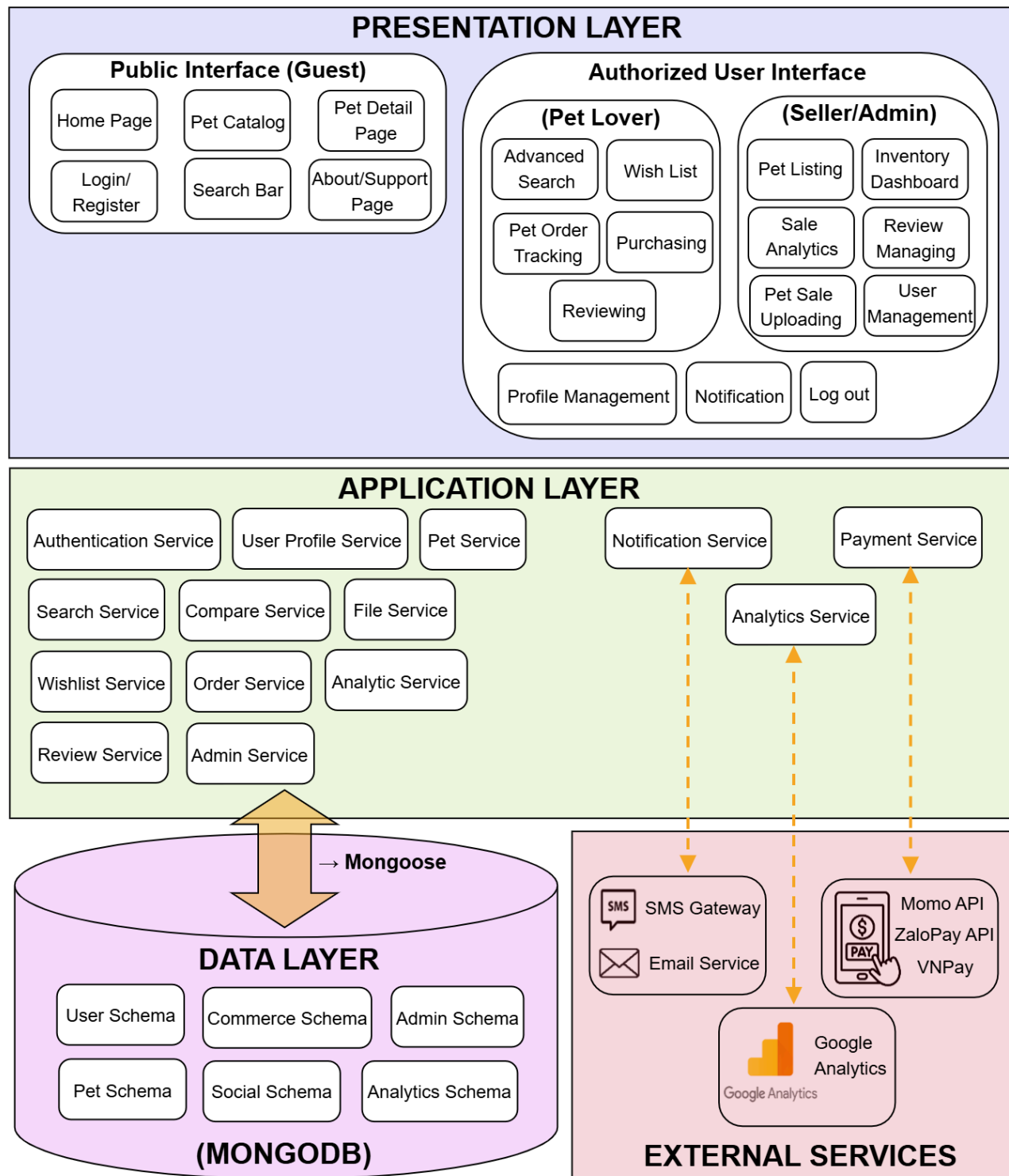
There are three main **user roles** in the system — **Pet Love**, **Registered User with All Users**, and **Admin** — each with specific demands and requirements that shape the system's features:

User Role	Demand	Functional Requirement	Non-Functional Requirements
Pet Lover	As a pet lover, I want to find pets that match my preferences and budget	Implement search and filtering by price, breed, age, size, and availability.	Search results load within 2s, responsive interface
	I want to compare pets by price, breed, and age, so that I can make an informed choice	Provide a comparison tool or table view showing multiple pets side by side with relevant attributes.	Comparison view should be intuitive and mobile-friendly
	I want to view detailed profiles of pets, so that I can learn about their personality, health, and care requirements	Create detailed pet pages with photos, descriptions, health info, age, breed, and care instructions.	Pet profile pages should have optimized image loading
	I want to save pets to a wishlist and receive notifications, so that I can track pets I'm interested in	Implement wishlist/favorites functionality and notification system for updates.	Notifications should be real-time and configurable
	I want to complete secure online purchases, so that I can buy pets safely and conveniently	Integrate secure payment gateway (credit/debit cards, e-wallets) and checkout process.	Must comply with PCI DSS standards, use HTTPS/SSL

	As a pet lover, I want to read reviews and ratings from other buyers, so that I can trust the quality and health of the pet before making a decision	Implement a review and rating system for pets, allowing buyers to leave feedback, rate their experience, and display average ratings on pet profiles	Reviews moderated for inappropriate content
Registered User	As a registered user, I want to manage my account details, so that my personal information and preferences are up to date.	Implement user profile management: edit personal info, change password, manage preferences.	Passwords hashed using bcrypt, responsive account UI
	As a registered user, I want to track my orders and order history, so that I can see the status of my purchases.	Provide order history page with tracking and status updates.	Order data cached for quick retrieval
All Users	I want to log in, register	Implement authentication system with verification email	Secure login (JWT + HTTPS), protect against SQLi/XSS
	I want the site to work smoothly on any device	Ensure responsive design and intuitive UI/UX	Support mobile, tablet, desktop; follow WCAG 2.1
	I want the system to be secure and reliable	Regular backups, encrypted connections, session timeouts	9.5% uptime, daily database backups, fail-safe recovery
	As an admin, I want to approve or remove pet listings, so that the platform maintains quality and safety standards.	Implement an admin panel to approve, reject, or delete pet listings.	Only verified admins can perform moderation actions
	As an admin, I want to manage user roles and monitor activity, so that the system runs smoothly and securely.	Provide admin functionality for user role management, account suspension, and activity monitoring.	All admin actions recorded for auditability

Admin/ Seller	As an admin, I want to access analytics about users and pets, so that I can make data-driven improvements.	Implement a reporting dashboard with statistics on users, pets, orders, and platform usage.	Dashboard loads within 3s; charts updated in real time
	As an admin, I want to list pets for sale with photos and details, so that potential buyers can see accurate information.	Implement a pet listing feature: add, edit, delete pets, upload photos, and input details.	Image uploads $\leq$ 2MB per file; validation on all fields
	As an admin, I want to manage my pet inventory, so that I know which pets are available or sold.	Provide an inventory management dashboard showing all pets and their status.	Must support large datasets ($\geq$ 10,000 entries)
	As an admin, I want to receive messages from interested buyers, so that I can answer questions and increase sales.	Implement a messaging or contact system for buyer-seller communication.	Notifications sent instantly; prevent spam messages
	As an admin, I want to see sales and performance reports, so that I can track my business growth.	Provide sales analytics and performance reports with charts and statistics.	Data updated daily; charts rendered $<$ 2s

3.1.2. Software Architecture



❖ Presentation Layer

The presentation Layer is responsible for all user interactions. It defines how users access and experience the system through web interfaces. It sends user requests to the Application Layer and displays the processed results back to users.

We develop using React frameworks and Tailwind CSS frameworks to ensure responsiveness and user interactivity

1. Public Interface (Guest): Accessible without login; provides general browsing and information.

- **Home Page** – Introduces the platform and features pets.
- **Pet Catalog** – Displays all available pets with filters and sorting.
- **Pet Detail Page** – Shows detailed information of a selected pet.
- **Login/Register** – Handles user authentication and account creation.
- **Search Bar** – Allows keyword-based search across pets.
- **About/Support Page** – Provides help, FAQs, and contact information.

2. Authorized User Interface: For registered users with different roles.

- **Pet Lover**
 - **Advanced Search** – Enables refined filtering (breed, price, age, etc.).
 - **Wish List** – Stores favorite pets.
 - **Pet Order Tracking** – Tracks order and delivery status.
 - **Purchasing** – Handles checkout and payment flow.
 - **Reviewing** – Allows users to rate and review sellers or pets.
- **Admin/Seller**
 - **Pet Listing** – Allows posting and editing of pets for sale.
 - **Inventory Dashboard** – Manages current listings and sales performance.
 - **Sale Analytics** – Provides insights into sales and pet popularity.
 - **Review Managing** – Monitors and responds to user reviews.
 - **Pet Sale Uploading** – Supports uploading pet images and information.

All authorized user can be able to:

- **Profile Management** – Edits personal info, addresses, and preferences.
- **Notification** – Displays order updates or promotions.
- **Log out** – Ends current session securely.

❖ Application Layer

The Application Layer encapsulates all business logic.

It acts as the middle layer that processes requests from the Presentation Layer, applies business rules, communicates with the database through Mongoose, and integrates with external services.

Key Services

- **Authentication Service** – Manages login, registration, token generation, and authorization using JWT.
- **User Profile Service** – Handles CRUD operations on user data and preferences.
- **Pet Service** – Manages pet-related operations such as listing, searching, and updating pet info.
- **Search Service** – Performs search queries across pet listings using keywords or filters.
- **Compare Service** – Allows users to compare different pets by attributes.
- **Wishlist Service** – Saves and retrieves the user's saved pets.
- **Order Service** – Handles the order lifecycle (placing, updating, canceling orders).
- **Review Service** – Stores and retrieves user reviews and ratings.
- **File Service** – Manages pet image uploads and file storage.
- **Analytic Service** – Gathers and aggregates data for admin dashboards or seller analytics.
- **Notification Service** – Sends updates via SMS or email.
- **Payment Service** – Processes transactions via integrated payment gateways.
- **Admin Service** – Supports administrative actions like monitoring, banning users, or system stats.

❖ Data Layer

The Data Layer is responsible for data storage, retrieval, and persistence. It abstracts database interactions and ensures data consistency through defined schemas and models.

1. Database: MongoDB

- A **NoSQL** database used for flexible document-based storage.
- Data is organized into **collections** instead of tables.

2. ORM: Mongoose

- Mongoose acts as a bridge between the Application Layer and MongoDB.
- It defines **schemas** and **models** to structure and validate data.
- Provides a high-level API for CRUD operations.

3. Schemas: Each schema corresponds to a major entity in the system:

- **User Schema** – Stores login info, roles, and profiles.
 - **Pet Schema** – Contains pet details such as name, breed, age, price, and seller reference.
 - **Commerce Schema** – Holds order, payment, and transaction data.
 - **Admin Schema** – Defines admin accounts, roles, and system settings.
 - **Social Schema** – Manages reviews, wishlists, and user interactions.
 - **Analytics Schema** – Logs user activities and sales for reporting and metrics.
- ❖ **External Services Layer:** This layer connects the system with third-party APIs to extend functionality and support advanced features like payments, notifications, and analytics.

External Integrations

- **Payment Gateways**
 - **MoMo API, ZaloPay API, VNPay API** – Handle secure online payments.
 - Integrated with the **Payment Service** in the Application Layer.
- **Analytics Tools**
 - **Google Analytics** – Tracks user behavior, traffic sources, and engagement data.
 - Connected to the **Analytic Service** for generating reports.
- **Communication Channels**
 - **SMS Gateway** – Sends OTPs and transaction updates to users.
 - **Email Service** – Sends invoices, confirmation messages, and notifications.

3.2 Hardware

3.2.1. Features

- Users access the application through a modern web browser. Therefore, the hardware requirements include:
 - Computer, laptop.

- Minimum RAM: 2GB
- Browser supporting HTML5 (Chrome, Firefox, Microsoft Edge)
- Stable Internet connection

3.2.2. Development Environment:

- To develop, test, and deploy the system, each team member needs to use a personal computer with the following configuration:
 - Operating system: Windows, macOS, or Linux.
 - RAM: Minimum 8GB (16GB recommended to run multiple tools simultaneously)
 - Hard drive: $\geq 100\text{GB}$ free (store data)
 - High-speed Internet connection
- Software to be installed:
 - Visual Studio Code (or equivalent IDE).
 - Node.js, npm (for frontend)
 - Python (Flask)
 - Git (source code management)
 - MongoDB Compass or MongoDB Atlas connection

3.2.3. Server deployment (Hosting & Database Server)

- The system will be deployed on a free platform suitable for students:
 - **Render or Vercel:** Used to deploy Flask backend or React frontend
 - **MongoDB Atlas (Free Tier):** Store data on NoSQL cloud platform
- Because of using free cloud services, there is no need to invest in separate server hardware, helping to save costs and facilitate deployment.

4

Development Plan

You can click this link to see the development plan of our team: [Link The Development Plan](#)

4.1 Requirements Analysis

- **Objective:** Understand the needs of each user group (Pet Lover, Seller, Admin, Registered User) to determine the main functions of the system.
- **Deliverables:**
 - Software Requirement Specification (SRS)
 - Use Case Diagram, User Stories List
- **Assignment:**

Task	Deadline	Participants
Gather and analyze project requirements	27/10/2025	Nhat Khoi, Gia Bao, Hai Dang, Quoc Duy, Truong Nguyen
Write documentation: Development Plan and Tool Setup	01/11/2025	Quoc Duy, Hai Dang
Write the Preliminary Problem Statement	01/11/2025	Gia Bao

4.2 Software Design

- **Objective:** Design the overall and detailed architecture for the system, ensuring scalability and maintainability.
- **Deliverables:**
 - Software Design Document
 - UML diagrams (class, sequence)
 - UI/UX Wireframes and Mockups (Figma or similar)
- **Assignment:**

Task	Deadline	Participants
Identify User Roles (Customer, Admin, Seller)	27/10/2025	Quoc Duy
Create User Stories and Use Case Diagram	29/10/2025	Gia Bao, Quoc Duy
Draw Class Diagram, Sequence Diagram, and Activity Diagram.	30/10/2025	Nhat Khoi, Quoc Duy
Design UI Wireframes for customer and admin interfaces (Figma or similar)	01/11/2025	Nhat Khoi, Quoc Duy
Define Software Architecture (React + Flask + MongoDB)	02/11/2025	Hai Dang
Set up development tools: Trello, GitHub, Discord, MongoDB Atlas, and Render	03/11/2025	Hai Dang, Truong Nguyen

4.3 Implementation

- **Objective:** Develop the system based on the design using appropriate technologies and frameworks.
- **Deliverables:**
 - Complete client-side user interface
 - Fully functional backend with CRUD API
 - Admin dashboard functionality
- **Assignment:**

Task	Deadline	Participants
Integrate basic API calls (fetch from Flask mock API)	20/11/2025	Gia Bao
Frontend Development (Client Side)		
Initialize React + TailwindCSS project	06/11/2025	Truong Nguyen

Implement Home Page and Navigation Bar (Figma or similar)	08/11/2025	Quoc Duy
Build Pet Listing & Detail Pages	10/11/2025	Hai Dang, Truong Nguyen
Add Search & Filter (by species, price, location).	12/11/2025	Nhat Khoi
Create Login & Register pages	13/11/2025	Truong Nguyen, Quoc Duy
Add Shopping Cart & Favorites features	16/11/2025	Truong Nguyen
Responsive layout for all pages	19/11/2025	Quoc Duy
Push & review frontend code on GitHub	19/11/2025	Truong Nguyen
Backend & Admin Dashboard		
Set up Flask backend project	21/11/2025	Hai Dang
Connect MongoDB database (models: Users, Pets, Orders, Categories)	22/11/2025	Nhat Khoi
Apply password encryption and secure routing	03/12/2025	Hai Dang
Implement Admin Authentication (JWT)	26/11/2025	Nhat Khoi, Hai Dang
Implement image upload	01/12/2025	Gia Bao
Develop Admin Dashboard UI with: Manage pets (add/edit/delete), Manage users, Manage categories/orders	28/11/2025	Gia Bao, Quoc Duy
Integrate Landing page with backend API	01/12/2025	Truong Nguyen
Create category tree with	03/12/2025	Truong Nguyen

backend API		
Integrate Category list with backend API	03/12/2025	Truong Nguyen, Gia Bao
Integrate Detail page with backend API	04/12/2025	Gia Bao
Handle wishlist	05/12/2025	Hai Dang, Quoc Duy
Handle cart, payment	05/12/2025	Nhat Khoi

4.4 Testing

- **Objective:** Verify that the system functions correctly, meets requirements, and is free from critical bugs.
- **Deliverables:**
 - Testing documents (Test Plan, Test Case)
 - Test report with results.
- **Assignment:**

Task	Deadline	Participants
Write a test plan & test	05/12/2025	Gia Bao
Fix bugs and retest	05/12/2025	Gia Bao
Perform unit testing (Flask & React components)	08/12/2025	Gia Bao, Hai Dang
Fix bugs and retest	10/12/2025	Nhat Khoi, Gia Bao, Hai Dang, Quoc Duy, Truong Nguyen

4.5 Deployment and Maintenance

- **Objective:** Deploy the system to a live environment and ensure stable and secure operation post-launch.
- **Deliverables:**
 - Live deployment link
 - User manual and usage guide
 - System maintenance documentation
- **Assignment:**

Task	Deadline	Participants
Deploy project to render/vercel	14/12/2025	Hai Dang, Truong Nguyen
Write the final project report	16/12/2025	Gia Bao
Complete weekly progress reports	17/12/2025	Gia Bao
Prepare the user manual and final presentation	19/12/2025	Hai Dang, Quoc Duy
Archive all deliverables in /pa and /docs	21/12/2025	Nhat Khoi, Gia Bao, Hai Dang, Quoc Duy, Truong Nguyen

5

Human Resources & Costing Plan

5.1 Human Resources

- The 5Pets project is developed by a team of 5 members, each taking responsibility for specific roles across the software development life cycle.
- The table below summarizes the personnel structure and key contributions:

Member	Responsibilities	Roles
Nhat Khoi	System analysis, database modeling, backend API development (Flask, MongoDB)	Database & Integration
Gia Bao	Requirements documentation, testing (unit/integration), project report writing	Documentation & Testing

Hai Dang	Backend setup, authentication, system architecture, integration, and deployment	Backend Lead
Quoc Duy	UI/UX design (Figma), frontend component development (React), admin dashboard	UI/UX
Truong Nguyen	React project setup, frontend development, shopping cart, wishlist, and responsiveness	Frontend Dev

- The team follows the **Waterfall** methodology with clearly defined roles and responsibilities. Collaboration and progress tracking are managed through Trello and Discord, with reviews conducted at the end of each development phase.

5.2 Cost Estimation Plan

- Since this project is developed in an academic environment and leverages many open-source tools and free-tier cloud platforms, the anticipated cost is minimal.
- The cost estimation is shown below:

Item / Service	Platform or Tool Used	Estimated Cost (USD)
Frontend Hosting	Vercel (Free Plan)	\$0
Backend Hosting	Render (Free Tier), Vietnix (Paid)	\$0 - \$12

Database	MongoDB Atlas (Free Tier)	\$0
UI Design	Figma (Free Plan)	\$0
Domain Name (Optional)	Freenom / GitHub Pages / Vietnix / Others	\$0 - \$10
Development Tools	VS Code, Git, Discord, Trello	\$0
Documentation & Testing	Google Docs, GitHub, Markdown	\$0

- **Total Estimated Cost: - \$0 – \$22 USD**
- This low-cost structure is achieved by carefully selecting free-tier tools and platforms suitable for academic and small-team development.

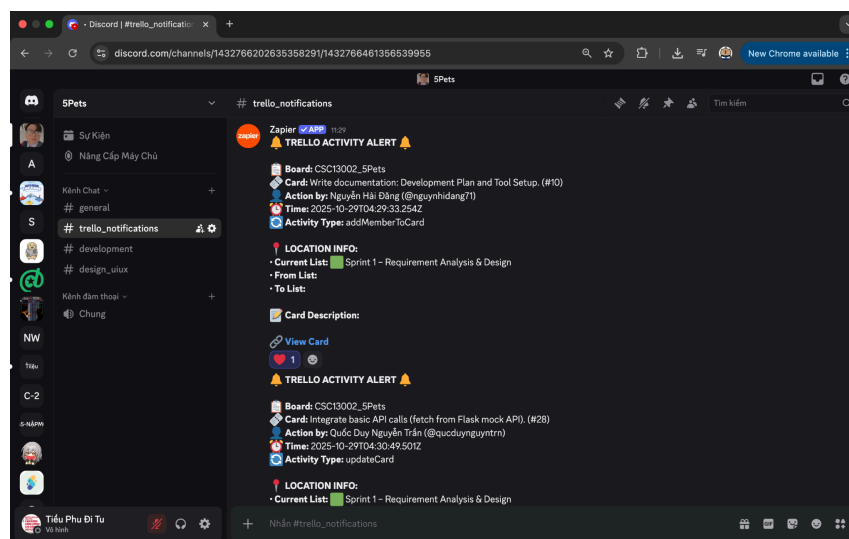
6

Tools setup

To ensure effective teamwork and organized project management throughout the **5Pets** development process, our team established a consistent set of collaboration and development tools. These tools support communication, task tracking, source control, and assignment submission in alignment with the **Waterfall model**.

6.1 Communication and Collaboration

- **Discord Server:**
 - Server name: 5Pets
 - Channels:
 - **#general** – for group-wide announcements and discussions
 - **#development** – for daily technical discussions and progress updates
 - **#design_uiux** – for sharing UI mockups and design feedback
 - **#trello_notifications** – automatically synced with Trello to notify task status changes
 - All members use both desktop and mobile apps to ensure consistent communication and quick responses.
 - **Discord** serves as the main hub for collaboration, replacing Slack.



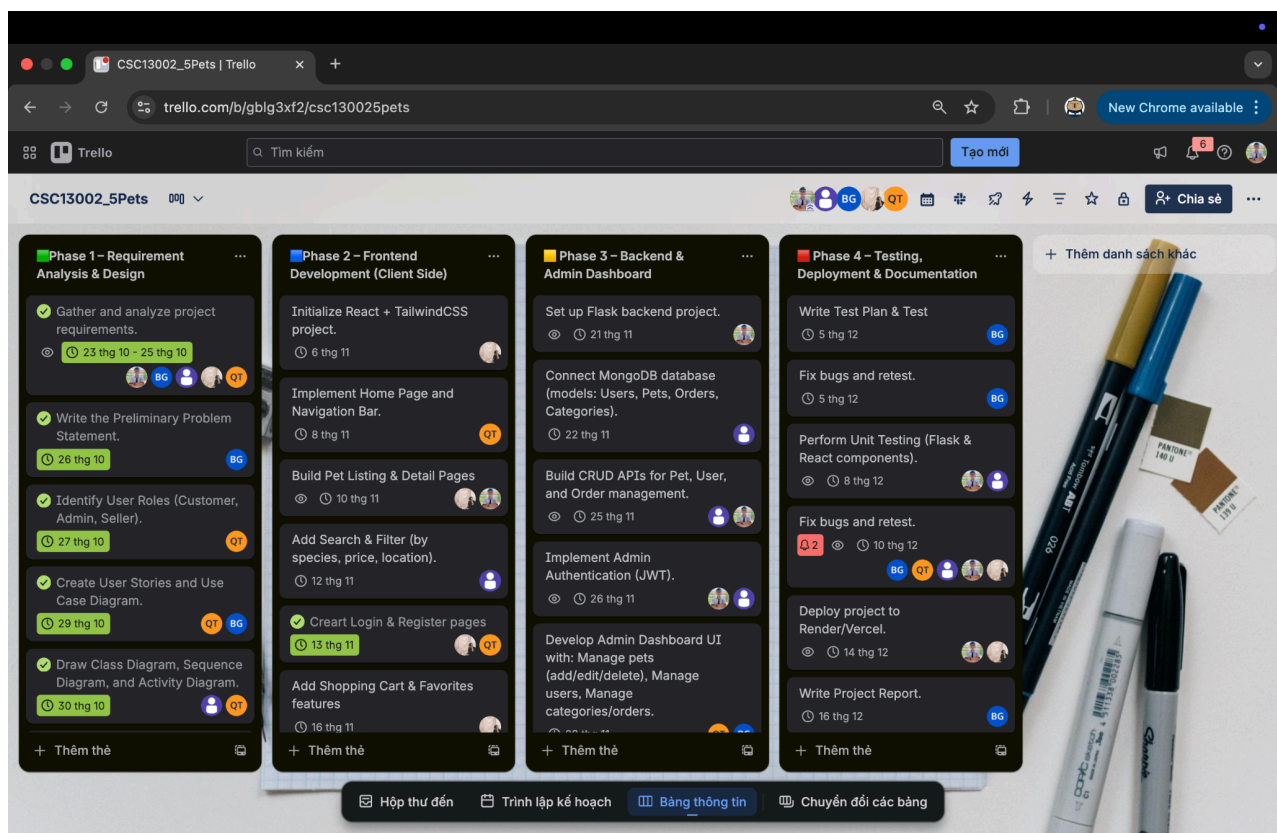
6.2 Project Management

- Trello Board:

The project board, named **CSC13002_5Pets**, follows the **Waterfall model**. Tasks are divided into four sequential phases, corresponding to key stages of the software lifecycle:

1. **Phase 1 – Requirement Analysis & Design**
(Use case diagrams, class diagrams, architecture, tool setup)
2. **Phase 2 – Frontend Development (Client Side)**
(React + TailwindCSS implementation, navigation, and UI features)
3. **Phase 3 – Backend & Admin Dashboard**
(Flask + MongoDB integration, CRUD APIs, admin authentication, dashboard UI)
4. **Phase 4 – Testing, Deployment & Documentation**
(Testing, deployment to Render/Vercel, final report, and presentation)

Each Trello card represents a task with assigned members, due dates, and detailed checklists. Trello is integrated with Discord to automatically notify the team whenever a card is updated or completed.



6.3 Source Control

- **GitHub Repository:**

Repository name: **5Pets**

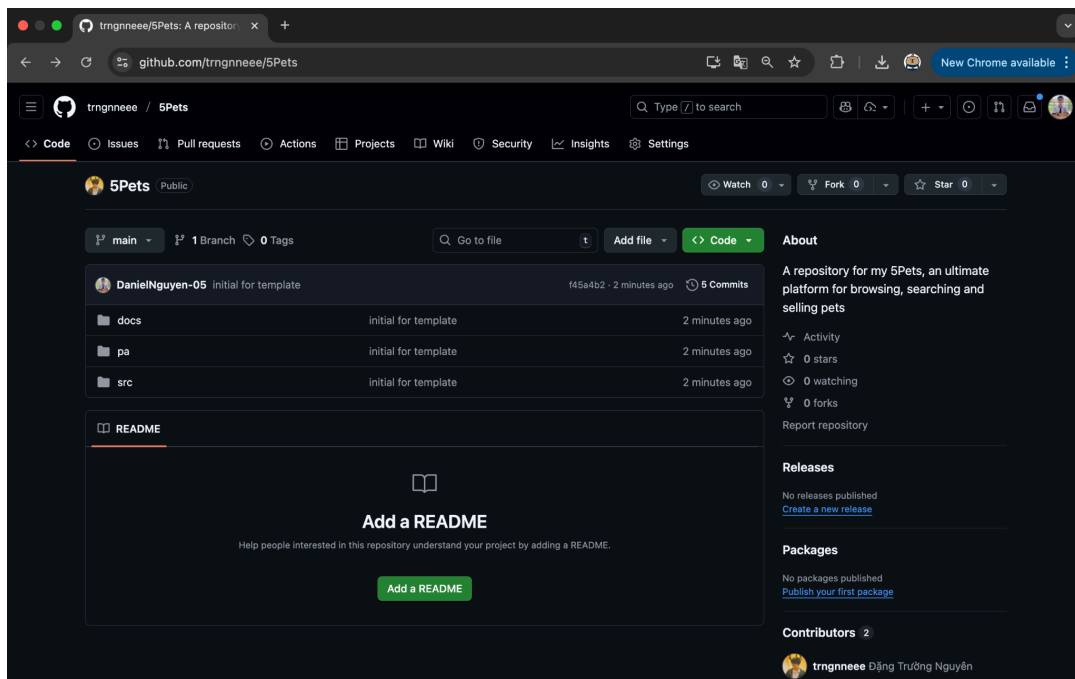
Repository structure:

/src	→ Source code (frontend & backend)
/docs	
/management	→ Planning documents and reports
/requirements	→ Vision document, use cases, requirements
/analysis_design	→ Architecture, UML diagrams, UI mockups
/test	→ Test plans, test cases, and test reports
/pa	→ Project assignment folders (PA0, PA1, ...)

- **Branching Strategy:**

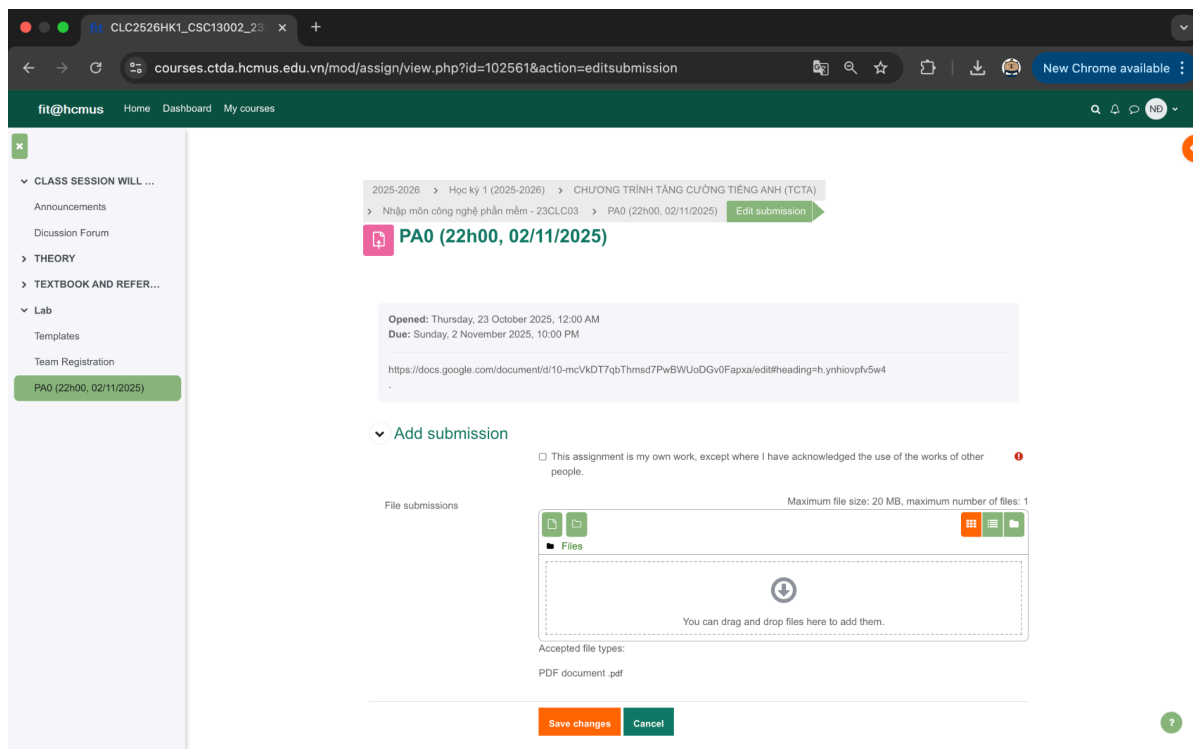
- **main** → stable release
- **name_dev** → active development for each member
- **feature/*** → new feature branches

All commits include descriptive messages referencing corresponding Trello tasks. GitHub ensures version control, code integrity, and transparent contribution tracking among all members.



6.4 Assignment Management

- **Moodle:**
Used for submitting project assignments (PAO, PA1, etc.) and receiving feedback from instructors. Each submission is also stored in the `/pa` folder of the GitHub repository.



6.5 Monitoring and Evaluation

- Team members are required to:
 - Update their Trello cards regularly.
 - Communicate daily via Discord.
 - Commit code weekly to GitHub.
- Instructors and TAs can monitor progress via Trello–Discord integration and GitHub activity logs.